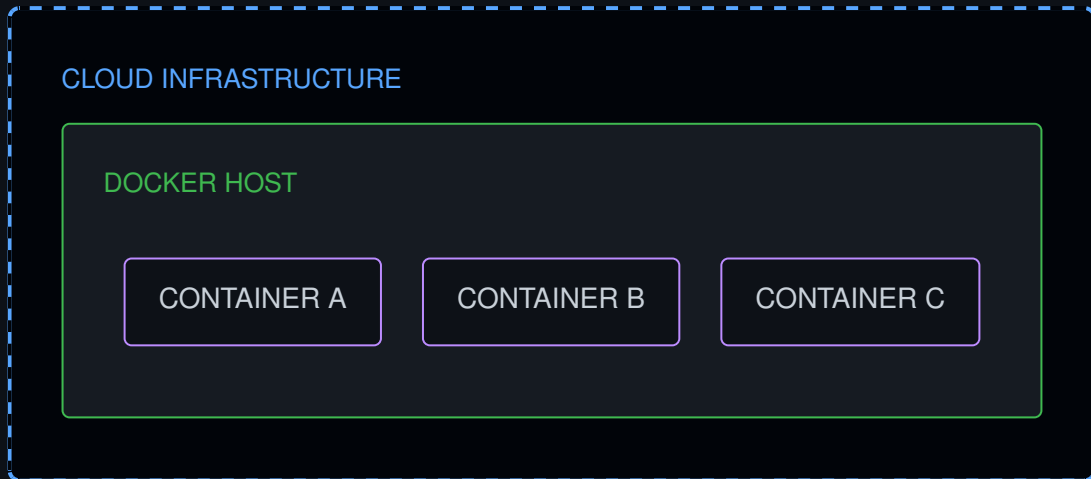


2026 EDITION

DOCKER ROADMAP 2026

From Containers to Production-Ready Engineering

LEARN • BUILD • DEPLOY • SCALE



// **ALGOTUTOR**

How to Use This Roadmap

Welcome to the **Docker Roadmap 2026**. This guide is designed to take you from absolute beginner to a production-ready engineer.

Roadmap Philosophy

Do not just memorize CLI commands. Follow this progression to build deep, practical expertise:



Skill Levels

BEGINNER

Core concepts, local setup, basic commands, and running your first containers. Essential for everyone.

INTERMEDIATE

Writing custom Dockerfiles, multi-container apps (Compose), networking, and persistent storage.

ADVANCED

Optimization, debugging, multi-stage builds, CI/CD integration, and custom registries.

PRODUCTION

Security hardening, resource constraints, zero-downtime scaling, and cloud/K8s readiness.

STAGE 0 — Prerequisites

"You don't need to master Linux before Docker — but you should be comfortable with the basics."

Linux Fundamentals

- Filesystem navigation (cd, ls, pwd)
- Permissions (chmod, chown)
- Processes (ps, top, kill)
- Environment variables (export, env)

Networking Basics

- IP addresses & Ports
- TCP vs UDP
- localhost vs `0.0.0.0`
- Reverse proxies (concept)

STAGE 1 — Docker Fundamentals

What is Docker? Docker is a platform that allows developers to package applications into standardized executable components called containers.

Virtual Machines (VM)

Heavyweight. Hardware-level virtualization. Each VM runs a full Guest OS (GBs in size). Slow boot time.

VS

Containers

Lightweight. OS-level virtualization. Shares the Host OS kernel (MBs in size). Instant boot time.

Docker Architecture

Client (Docker CLI)

docker build, pull, run

⇔

Docker Host (Daemon)

Manages Containers & Images

⇔

Registry (Docker Hub)

Stores Images

The core flow: **Dockerfile** → **Build** → **Image** → **Run** → **Container**

STAGE 2 — Docker CLI Reference

You don't need to memorize every flag, but these commands are your daily toolkit.

Command	Purpose	Real-world Example
<code>docker pull <image></code>	Downloads an image from a registry.	<code>docker pull ubuntu:22.04</code>
<code>docker images</code>	Lists all locally stored images.	<code>docker images -a</code>
<code>docker run <image></code>	Creates and starts a new container.	<code>docker run -d -p 80:80 nginx</code>
<code>docker ps</code>	Lists running containers.	<code>docker ps -a</code> (shows stopped too)
<code>docker stop <id></code>	Gracefully stops a running container.	<code>docker stop my-api-container</code>
<code>docker rm <id></code>	Removes a stopped container.	<code>docker rm -f web-app</code> (force remove)
<code>docker rmi <image></code>	Removes an image from local storage.	<code>docker rmi node:18-alpine</code>
<code>docker logs <id></code>	Fetches the logs of a container.	<code>docker logs -f my-db</code> (-f to follow)
<code>docker exec -it <id> sh</code>	Runs a command in a running container.	<code>docker exec -it web-app bash</code>
<code>docker inspect <id></code>	Returns low-level info (JSON).	<code>docker inspect my-container</code>
<code>docker system prune</code>	Cleans up unused data.	<code>docker system prune -a --volumes</code>

STAGE 3 — Docker Images

A Docker image is a read-only template containing instructions for creating a Docker container. Images are immutable.

Image Anatomy

- **Layers:** Each instruction in a Dockerfile creates a layer.
- **Tags:** Used for versioning (e.g., `node:18`).
- **Immutable:** Cannot be changed after creation.

Layer 4 (App Code)

Layer 3 (Dependencies)

Layer 2 (Runtime)

Layer 1 (Base OS - Alpine)

Caching works bottom up!

STAGE 4 — The Dockerfile

Example: Node.js API

```
# 1. Base Image
FROM node:18-alpine

# 2. Set working directory inside container
WORKDIR /usr/src/app

# 3. Copy package files first (leverage layer caching)
COPY package*.json ./
RUN npm ci --only=production

# 4. Copy the rest of the application code
COPY . .

# 5. Document the port
EXPOSE 3000

# 6. Define the runtime command
CMD ["node", "server.js"]
```

CMD vs ENTRYPOINT	<code>CMD</code> provides defaults and can be overridden. <code>ENTRYPOINT</code> sets the container to run as a specific executable.
COPY vs ADD	Use <code>COPY</code> for standard files. <code>ADD</code> can download from URLs and extract archives (discouraged).

STAGE 5 — Building Efficient Images

ADVANCED

PRODUCTION

Bad Practice (Bloated)

```
FROM ubuntu:latest
WORKDIR /app
COPY . .
RUN apt-get update && apt-get install -y nodejs npm
RUN npm install
CMD ["node", "app.js"]
```

- Uses huge OS base image.
- Copies code before installing deps (breaks cache).
- Runs as root user.

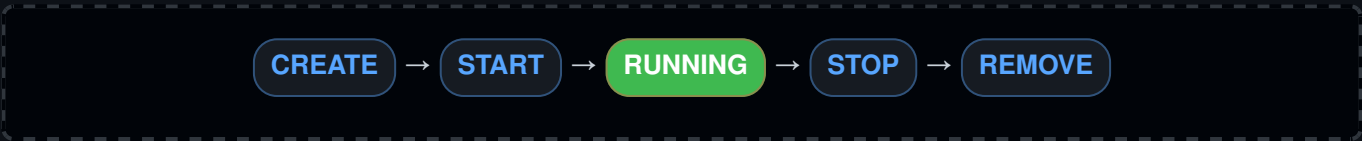
Optimized Practice

```
FROM node:18-alpine
WORKDIR /app
COPY package.json ./
RUN npm ci
COPY . .
USER node
CMD ["node", "app.js"]
```

- Minimal Alpine base.
- Optimal caching order.
- Non-root USER for security.
- Uses `.dockerignore`.

STAGE 6 — Docker Containers

A container is a running instance of an image.



Port Mapping	<code>-p 8080:80</code> maps Host port 8080 to Container port 80.
Env Variables	<code>-e DB_PASS=secret</code> passes configuration safely.
Detached Mode	<code>-d</code> runs the container in the background.
Restart Policies	<code>--restart unless-stopped</code> ensures auto-reboot on crash.

STAGE 7 — Persistent Data

INTERMEDIATE

Containers are ephemeral. Data that must survive deletion requires external storage.

Storage Type	Best For	Typical Usage
Named Volumes Managed by Docker	Database Persistence	<code>-v pgdata:/var/lib/postgresql/data</code> . Docker manages host location. Recommended for DBs.
Bind Mounts Managed by User	Local Development	<code>-v \$(pwd) :/app</code> . Maps a specific host directory. Great for live-reloading.
tmpfs Mounts In-Memory	Security / Speed	<code>--tmpfs /app/cache</code> . Data is stored in host RAM, never written to disk.

STAGE 8 — Docker Networking

Networking enables containers to talk to each other securely.

Bridge (Default)

User-defined bridges enable internal DNS (communicate via container name).

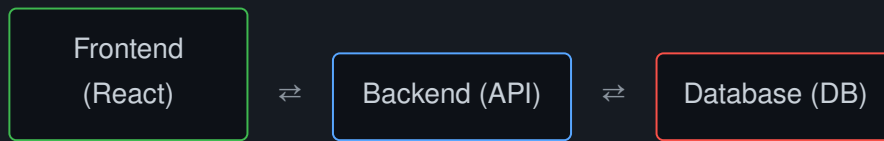
Host

Removes network isolation. Shares host networking namespace directly.

None

Complete isolation. The container has no network interfaces.

Secure Network Architecture



Important: Only the Frontend exposes port 80 to the public.
Backend and DB communicate purely over the internal Docker network.

STAGE 9 — Docker Compose

INTERMEDIATE

ADVANCED

Compose defines and runs multi-container applications using a single `compose.yaml`.

```
version: '3.8'

services:
  api:
    build: ./api
    ports:
      - "8080:8080"
    environment:
      - DB_URL=postgres://user:pass@db:5432/app
    depends_on:
      - db
    networks:
      - app-net

  db:
    image: postgres:15-alpine
    environment:
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=pass
      - POSTGRES_DB=app
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - app-net

volumes:
  pgdata:
networks:
  app-net:
```

Compose Commands

- `docker compose up -d`
Start all services in background.
- `docker compose down`
Stop & remove containers/networks.
- `docker compose logs -f`
Tail logs for all services.
- `docker compose build`
Force rebuild custom images.

STAGE 10 — Multi-Container Projects

Project	Stack	Key Docker Skills Practiced
Project 1 BEGIN	Static HTML + Nginx	Basic Dockerfile, COPY, Port mapping.
Project 2 BEGIN	Node.js REST API	WORKDIR, npm ci, Env variables.

Project	Stack	Key Docker Skills Practiced
Project 3 INTER	Python API + Redis	Docker Compose, Networks, DNS discovery.
Project 4 INTER	React + Node + Mongo	Multi-container orchestration, Volumes.
Project 5 ADV	Full Stack + Proxy	Multi-stage builds, Reverse Proxy.

STAGE 11 — Docker + Databases

INTERMEDIATE

ADVANCED

Crucial Distinction: Running a database in Docker for **local development** is an industry standard. Running a database in Docker for **production** requires deep expertise in volume management and IOPS tuning.

PostgreSQL / MySQL

Require explicit environment variables (e.g., root passwords). Mount volumes to `/var/lib/...` to prevent data loss.

Redis / MongoDB

Often run natively out of the box. Redis is entirely in-memory by default, making it easy to scale.

STAGE 12 — Docker Security

PRODUCTION

Security Checklist

- ✓ Run as non-root (`USER node`)
- ✓ Use trusted base images
- ✓ Scan images for CVEs
- ✓ Avoid secrets in Dockerfiles
- ✓ Use Read-Only filesystems
- ✓ Set CPU/Memory limits

Secrets Management

Never use `ENV` for sensitive data like API keys in a Dockerfile. Use Docker Secrets or pass them at runtime via `--env-file`.

Image Vulnerability Scanning

Integrate scanning tools (Docker Scout, Trivy) into your CI/CD pipeline to block deployments of vulnerable images.

STAGE 13 — Debugging & Troubleshooting

Observe → Logs → Inspect → Exec → Network Check → Fix

Symptom	Probable Cause & Command to Fix
Container exits immediately	Main process died. Check <code>docker logs <id></code> . Ensure <code>CMD</code> runs a foreground process.
Port already in use	Another service is using the host port. Change mapping to <code>-p 8081:80</code> .
Cannot connect to DB	Using <code>localhost</code> inside App container points to the App container. Change DB host to the service name (e.g., <code>db</code>).
Permission Denied on Volume	Host user UID/GID does not match container's UID/GID. Fix permissions on host.

STAGE 14 — Docker Performance

ADVANCED

Build Time

Structure Dockerfiles from least frequently changed (OS updates, deps) to most frequently changed (code).

Image Size

Use `alpine` or `distroless` images. Multi-stage builds strip compilers out of the final image.

Resource Limits

Use `--memory="512m" --cpus="1.0"` to prevent containers from eating 100% of host CPU/RAM.

STAGE 15 — Docker in CI/CD

Git Push → Build & Test (CI) → docker push → Registry → Deploy

STAGE 16 — Registries & STAGE 17 — Cloud

Registries

Docker Hub (public) vs AWS ECR / Azure ACR (private). Used to store and distribute images globally.

Cloud Platforms

AWS ECS/Fargate, Google Cloud Run, Azure ACI allow running containers without managing VMs.

STAGE 18 — The Kubernetes Bridge

"Docker helps you build and package containers. Kubernetes helps you orchestrate and scale them."

Docker (Single Node)

`docker run`
Docker Compose
Manual scaling

→

Kubernetes (Cluster)

Pods & Deployments
Auto-scaling (HPA)
Self-healing

Production Skills & Resources

Category	Beginner	Intermediate	Advanced	Production
Dockerfile	FROM, RUN, CMD	ENV, EXPOSE, WORKDIR	Multi-stage, ARG	Distroless, USER, HEALTHCHECK
Networking	Port Mapping (-p)	Bridge, Compose DNS	Custom Subnets, Host	Network Policies, Service Mesh
Storage	Ephemeral FS	Named Volumes, Bind	tmpfs mounts	CSI drivers, Backups

The Job-Ready Checklist

☐ Run containers confidently

☐ Write production Dockerfiles

☐ Optimize image size & layers

☐ Use Docker Compose for stacks

☐ Configure internal networks

☐ Manage persistent volumes

☐ Containerize DBs for dev

☐ Apply Docker security practices

☐ Build CI/CD pipelines

☐ Push to custom registries

KEEP BUILDING. KEEP LEARNING. KEEP SHIPPING.

Your roadmap doesn't end with Docker.
It starts with the ability to build systems that actually run.

Docker → DevOps → Cloud → Kubernetes → Production Engineering

// ALGOTUTOR

Created for developers who want to build real-world skills — not just collect tutorials.

Follow AlgoTutor for practical roadmaps, interview guides, cheat sheets, career insights & developer resources

LEARN. BUILD. SHIP. GROW.